Self-Healing Infrastructure with Auto Scaling using AWS

Project Abstract and Report

1. Project Title

Self-Healing Infrastructure with Auto Scaling using AWS (EC2, Elastic Load Balancing, Auto Scaling and CloudWatch)

2. Abstract

Cloud computing provides scalable and reliable infrastructure for modern applications, but traditional systems still depend on manual monitoring to detect failures and handle changing workloads. This dependence leads to delayed recovery, unplanned downtime, and servers that are either over-provisioned or overloaded. This project implements a Self-Healing Infrastructure with Auto Scaling on Amazon Web Services (AWS). It uses Amazon EC2 for compute, Elastic Load Balancing (ELB) for traffic distribution, Auto Scaling Groups for capacity management, and Amazon CloudWatch for monitoring and alarms. The purpose of the project is to build an environment that automatically checks the health of its servers, replaces failed instances without human intervention, and adds or removes instances according to traffic demand. The expected outcome is a highly available application that keeps serving users even when an instance fails, maintains consistent performance during traffic spikes, and reduces cost by scaling down when demand is low. Overall, the project shows how automation can make cloud infrastructure more resilient, efficient, and easier to operate.

3. Introduction

Modern applications such as e-commerce sites, learning portals, and banking services are expected to be available around the clock. Even a few minutes of downtime can cause lost revenue, poor user experience, and damage to reputation. In traditional data centers and manually managed cloud setups, administrators must watch dashboards, detect failed servers, and launch replacements by hand. Traffic is also unpredictable: a sale, an exam result, or a viral post can multiply load within minutes, while at night the same servers sit mostly idle.

Cloud platforms like AWS solve this by offering services that can observe, decide, and act automatically. A self-healing system detects unhealthy components and replaces them on its own, while auto scaling adjusts the number of running servers to match demand. Together they provide high availability, stable performance, and efficient use of money and resources. This is why the topic is important for anyone learning cloud computing and DevOps practices.

4. Key Points of the Project

- **Automatic health monitoring:** The load balancer and CloudWatch continuously check whether each EC2 instance is healthy and responding.
- **Self-healing:** When an instance fails its health check, the Auto Scaling Group terminates it and launches a fresh replacement automatically.
- **Dynamic auto scaling:** Instances are added when metrics such as CPU utilization cross a high threshold and removed when load drops.
- **Load distribution:** Elastic Load Balancing spreads incoming requests across all healthy instances so that no single server is overloaded.
- **Multi-Availability-Zone deployment:** Instances run in more than one Availability Zone, so the application survives the failure of an entire zone.
- **Consistent configuration:** A launch template ensures every new instance has the same AMI, instance type, security groups, and startup script.
- **Alarms and notifications:** CloudWatch alarms trigger scaling actions and can notify administrators of important events.
- **Cost efficiency:** Pay-as-you-go pricing combined with scale-in during low traffic avoids paying for unused capacity.

5. Advantages and Limitations

Advantages

- **High availability:** Failed instances are detected and replaced automatically, minimizing downtime.
- **Better performance:** Extra capacity is added during peak traffic, so response times stay stable.
- **Cost savings:** Resources are reduced when demand falls, so only the required capacity is paid for.

- **Reduced manual effort:** Administrators no longer need to monitor servers continuously or react to failures by hand.
- **Scalability and flexibility:** The same design can grow from a few instances to many without changing the architecture.
- **Fault tolerance:** Spreading instances across Availability Zones protects against infrastructure-level failures.

Limitations

- **Configuration complexity:** Choosing correct thresholds, cooldown periods, and health check settings requires careful tuning.
- **Scaling delay:** New instances take time to boot and pass health checks, so very sudden spikes may briefly affect users.
- **Stateless design needed:** Applications that store session data or files on a single instance must be redesigned or use shared storage and databases.
- **Cost risks:** Wrong scaling rules can launch too many instances and increase the bill unexpectedly.
- **AWS dependency:** The solution relies on AWS-specific services, which makes moving to another provider harder.
- **Limited coverage:** Health checks detect instance and application failures, but they may not catch every kind of logical or data-level problem.

6. Components in the Diagram

- **Users / Clients:** End users who access the application through a browser or app. Their requests generate the traffic that the system must handle.
- **Elastic Load Balancer (ELB):** The single entry point for all traffic. It distributes requests across healthy EC2 instances and stops sending traffic to any instance that fails a health check.
- **Target Group and Health Checks:** The group of instances registered behind the load balancer. Periodic health checks, such as an HTTP request to a path, decide whether an instance is healthy or unhealthy.
- **Amazon EC2 Instances:** The virtual servers that run the application. They are spread across multiple Availability Zones for fault tolerance.

- **Launch Template:** Defines how each new instance is created, including the AMI, instance type, key pair, security groups, and startup script, so every replacement is identical.
- **Auto Scaling Group (ASG):** Maintains the desired number of instances between a minimum and maximum limit. It replaces unhealthy instances (self-healing) and launches or terminates instances based on scaling policies.
- **Amazon CloudWatch:** Collects metrics such as CPU utilization and request count, stores logs, and raises alarms when thresholds are crossed. These alarms trigger the scale-out and scale-in policies.
- **Scaling Policies:** Rules that tell the Auto Scaling Group what to do, for example adding an instance when average CPU stays above 70 percent and removing one when it falls below 30 percent.
- **VPC, Subnets and Security Groups:** The network layer that isolates the environment, places instances in different Availability Zones, and controls which traffic is allowed in and out.
- **Amazon SNS (optional):** Sends email or SMS notifications to administrators when scaling or failure events occur.

7. Future Scope

- **Predictive and scheduled scaling:** Use machine learning based predictive scaling or scheduled actions to add capacity before known traffic peaks.
- **Infrastructure as Code:** Define the whole setup using AWS CloudFormation or Terraform so it can be recreated consistently with one command.
- **Containers and Kubernetes:** Move the application to Amazon ECS or EKS to achieve faster scaling and better resource utilization.
- **Multi-region disaster recovery:** Extend the design across regions with Route 53 failover to survive a complete regional outage.
- **Advanced monitoring and security:** Add centralized logging, dashboards, AWS WAF, and automated remediation using Lambda functions.

8. Conclusion

This project demonstrates how AWS services can work together to build an infrastructure that monitors itself, repairs itself, and adjusts to demand without manual intervention. EC2 provides the compute capacity, Elastic Load Balancing distributes traffic and checks

health, the Auto Scaling Group replaces failed instances and manages capacity, and CloudWatch supplies the metrics and alarms that drive these decisions. The result is a system with high availability, better performance during peak load, and improved cost efficiency during quiet periods. Although careful configuration and a stateless application design are required, the benefits far outweigh the challenges. With future improvements such as predictive scaling, Infrastructure as Code, and multi-region deployment, the solution can grow into a production-grade, fully automated cloud platform.